

Research Article

A Structured Lifecycle Model for Quantum Software Engineering: Bridging Technical Challenges and Future Directions

Hessa Alfraihi ¹, Islam Zada ², Mahmoud Ahmed ³, Sara Shahzad ⁴, Siwar Rekik ⁵,
Abdullah Alshahrani ⁶, and Nguyen Phi ⁷

¹Department of Information Systems, College of Computer and Information Sciences, Princess Nourah Bint Abdulrahman University, Riyadh 11671, P.O. Box 84428, Saudi Arabia

²Department of Software Engineering, International Islamic University Islamabad, Islamabad 44000, Pakistan

³Faculty of Computer Studies, Arab Open University, A'Ali 732, Bahrain

⁴Department of Computer Science, University of Peshawar, Peshawar 25000, Pakistan

⁵Computer Science Department, College of Computer and Information Sciences, Prince Sultan University, Riyadh 11586, Saudi Arabia

⁶Department of Computer Science and Artificial Intelligence, College of Computer Science and Engineering, University of Jeddah, Jeddah 21493, Saudi Arabia

⁷Faculty of Engineering, Dong Nai Technology University, Dong Nai 76000, Vietnam

Correspondence should be addressed to Islam Zada; islam.zada@iiu.edu.pk and Nguyen Phi; nguyenp@dnvu.edu.vn

Received 24 September 2025; Revised 16 December 2025; Accepted 18 December 2025

Guest Editor: Zahid Akhtar

Copyright © 2026 Hessa Alfraihi et al. IET Software published by John Wiley & Sons Ltd. This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Quantum computing is one of the research areas progressing rapidly toward practical deployment, yet the engineering of scalable and reliable quantum software remains underdeveloped. Current quantum software engineering (QSE) practices are largely tools-driven and ad hoc that providing limited support for managing probabilistic execution, hybrid quantum–classical workflows, noise sensitivity, and hardware constraints. This study proposed a structured QSE lifecycle that integrates quantum-specific characteristics with disciplined software engineering practices and principles. The proposed lifecycle organizes development into six phases, encompassing quantum requirements engineering, formal modeling, architecture and circuit design, hybrid integration, noise-aware testing, and deployment with monitoring. Each phase is supported by explicit artifacts and quantitative criteria to enable systematic progression and iterative refinement. The QSE is validated through expert assessment and simulation-based experimentation using representative variational quantum algorithms under the realistic noise conditions. The results show improved fidelity convergence, reduced resource overhead, enhanced development stability (DS), and more reliable validation compared with unstructured workflows, demonstrating the value of lifecycle-driven engineering for quantum software systems.

Keywords: hybrid quantum–classical optimization; noise-aware quantum programing; probabilistic software verification; quantum software engineering (QSE); structured software lifecycle; variational quantum algorithms

1. Introduction

Quantum computing has become a paradigm shift in the world of computing, and it has the potential to solve the problem classes, which are still unsolvable by classical computing systems [1]. The use of continuous advancements in quantum

hardware, including a larger number of qubits, longer coherence times, and shorter gate error rates, has increased the pace of reaching large-scale quantum computation into practice [2, 3]. Consequently, quantum algorithms are also becoming a field of interest in the context of optimization, quantum chemistry, material science, cryptography, and machine

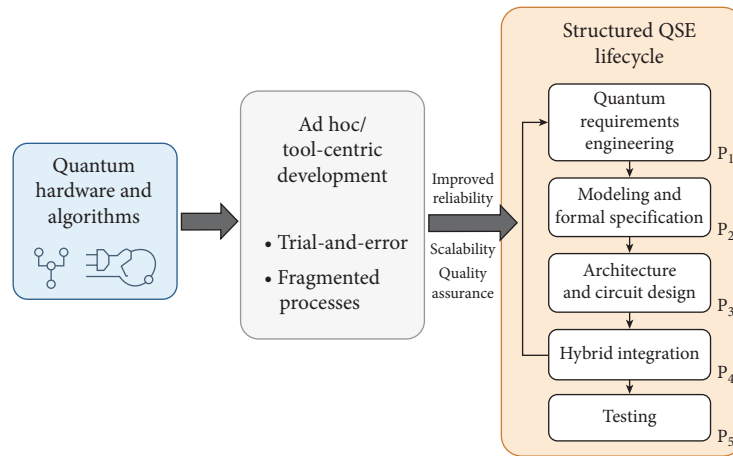


FIGURE 1: Conceptual motivation for a structured quantum software engineering lifecycle.

learning. Although these developments have been made, the engineering practices needed to be able to develop reliable and scalable quantum software systematically have not yet reached maturity.

The quantized software is fundamentally different from the classical software with respect to the execution semantics as well as the development constraints. Quantum programs are based on probabilistic principles and are strongly sensitive to noise; they require hardware-specific attributes such as qubit topology, decoherence, and gate fidelity. In addition, the majority of modern quantum applications are re-implemented using a hybrid quantum-classical implementation, in which classical optimizers repeatedly interact with quantum circuits by repeated measurements [4, 5]. The characteristics of these properties add complexity to conventional software engineering processes, including requirements elicitation, architectural design, testing and debugging, and deployment, thus direct use of classical development lifecycles is unsuitable [6, 7].

Practically, a large part of quantum software engineering (QSE) is now motivated by tool-centric and experimentation-based software development workflows [8]. Quantum software development kits are usually used to build and debug circuits by repeated trial and error with little formal advice on what phase to follow, how to handle artifacts, or quality control [9, 10]. Although these methods can be practical for small-scale experimental research, they are not effectively scalable to complicated or long-lived quantum systems. Lack of an organized engineering lifecycle increases the risk of development, decreases reproducibility, and makes maintenance more difficult in already noisy and hardware variable environments [11, 12].

The conceptual motivation for a structured approach to quantum software development has been demonstrated in Figure 1, which contrasts the accelerated pace of quantum hardware development and algorithm development against the ad hoc nature of the existing quantum software development. This absence of an explicit lifecycle has a disconnect between low-level quantum execution and high-level engineering objectives, including reliability, scalability, and systematic validation, as illustrated in the figure. The solution to this gap is

to have an engineering structure that clearly includes quantum-specific constraints at every stage of software development, starting with requirements and ending with deployment.

QSE is the new field that will solve these problems by extending software engineering concepts to quantum space [13, 14]. Existing QSE activities are, however, still disjointed and tend to concentrate on some isolated phases in the development or just offer high-level conceptual advice devoid of formal description, quantitative measures, and empirical demonstrations. It follows that a formal, holistic, and workable lifecycle model, which promotes rigorous quantum software development and allows probabilistic execution, noise, and hybrid workflows, is required.

The given paper suggests a limited-scope QSE lifecycle model that will overcome these constraints by structuring quantum software development into six mutually consistent phases. Both stages generate explicit artifacts, both phases are controlled by quantifiable constraints, and both phases aid in the process of progressively refining under realistic hardware constraints. Lifecycle transitions, optimization behavior, and testing outcomes are modeled mathematically, which allows a systematic quality evaluation and reproducibility. The primary objectives of this study are threefold:

- To establish a disciplined engineering structure for quantum software development that aligns with both quantum and classical components.
- To formalize lifecycle transitions using quantifiable metrics such as fidelity, error accumulation, and resource constraints.
- To demonstrate the practical effectiveness of the proposed lifecycle through expert assessment and simulation-based experimentation.

Contributions of this work are a detailed lifecycle model that is specific to the characteristics of quantum software systems, formal mathematical models of lifecycle phases and transitions, explicit hybrid quantum-classical workflows, an empirical evaluation framework of combining expert feedback and experimental analysis, and practical insights into

enhancing reliability, scalability, and reproducibility in quantum software development.

This conceptual overview presents the gap between rapid advancements in quantum hardware and algorithms and the predominantly ad hoc practices currently used in quantum software development. Figure 1 compares tool-based, trial-and-error, processes with a coordinated QSE lifecycle that integrates requirements analysis, formal modeling, architecture, and circuit design, hybrid quantum-classical execution, probabilistic testing, and deployment with continuous monitoring. The organized lifecycle emphasizes the role of explicit phase transitions and feedback over reliability, scalability, and quality assurance in quantum-specific limitations.

The rest of this paper is organized in the following way. Section 2 is a literature review of the underlying concepts of QSE and a critical analysis of the existing lifecycle models and research. Section 3 introduces the proposed structured QSE lifecycle along with its stages, formal definitions, and engineering artifacts. Section 4 explains the validation procedure and experimental testing, and results and analysis in Section 5. Lastly, in Section 6, the paper is concluded, and the future research directions are given.

2. Background and Related Work

QSE has been a response to the increased awareness that quantum algorithms and hardware development are not enough to achieve the development of reliable, scalable, and maintainable quantum applications. Although the theoretical algorithms and physical implementations of quantum computing have been studied extensively, the engineering-level of the process to convert these new achievements into viable software systems has not been developed sufficiently [8, 15]. It is an overview of the primary ideas in QSE, an overview of the major research works, and a critical assessment of the weaknesses in order to inspire the necessity of a structured lifecycle approach.

2.1. Foundations of QSE. The first quantum programming projects were algorithm-based, and aimed at establishing quantum advantage by isolated quantum computations of algorithms, including Shor factoring and Grover search [16, 17]. These attempts were critical of providing theoretical validation yet did not give much in terms of systematic development practices. As quantum software systems became more complex, science and engineering researchers started to consider higher-level abstractions, programming languages, and software toolkits in order to make quantum computer programming easier to develop.

Frameworks such as Qiskit, Cirq, Q# [18, 19], and PennyLane introduced standardized interfaces for constructing and executing quantum circuits, enabling rapid experimentation and hybrid quantum-classical workflows. Although these tools greatly reduced the barrier to quantum programming, they did not focus on organized engineering processes and focused more on convenient implementation. This has led to the common application of iterative experimentation without a clear lifecycle direction by developers, making them less scalable and reproducible [20].

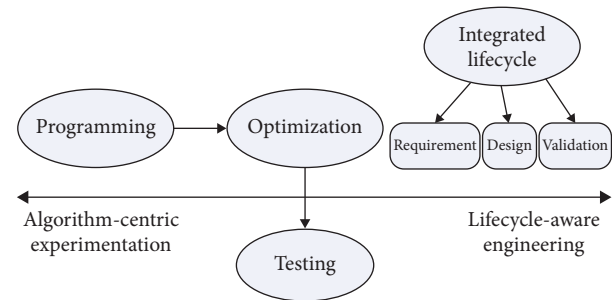


FIGURE 2: Evolution of quantum software engineering approaches.

Figure 2 defines the conceptual development of QSE and provides the way of how algorithm-centric experimentation advances to the lifecycle-aware engineering practices. As illustrated, the previous approaches would tend to focus on single steps of development, as opposed to providing a complete engineering system.

2.2. Lifecycle-Oriented Approaches in QSE. A number of authors have presented lifecycle-based views to structure quantum software development [10]. One of the earliest conceptual lifecycle models of QSE presented by Petersen and Piattini [6] was a variation of classical software development stages to quantum ones. They emphasized the use of quantum program requirements analysis, design, implementation, and testing in their model. Nevertheless, the method was predominantly descriptive without having formal mechanisms to tackle quantum specific limitations including noise, probabilistic outputs, or hybrid execution patterns.

Moguel et al. [21] extended this work by proposing a roadmap for QSE that emphasized organizational readiness, tooling ecosystems, and developer skills. Although their roadmap gave good high-level information, it was more focused on strategic adoption as opposed to specific engineering processes. Consequently, it provided shallow advice on the way the lifecycle phases were supposed to be implemented or assessed in practice. A more detailed study of QSE issues was provided by Zhao et al. [22], such as requirements engineering, verification, and maintenance. Their contribution presented a logical categorization of unresolved research issues and a powerful conceptual basis [23]. Nonetheless, it did not go as far as to suggest a lifecycle model that is integrated or to empirically validate the engineering process using empirical research. Collectively, these studies highlight the importance of structured development but remain limited by their reliance on conceptual descriptions without formalization or quantitative evaluation.

2.3. Hybrid Quantum-Classical Developmental Models. Hybrid quantum-classical computation has become central to near-term quantum applications, particularly for variational algorithms such as the variational quantum eigensolver (VQE) and the quantum approximate optimization algorithm (QAOA). A number of scholars have also studied the implications of software for hybrid execution, and how quantum circuits and classical optimization loops must be closely integrated [24–26].

TABLE 1: Comparison of existing quantum software engineering approaches.

Study/framework	Main focus	Strengths	Limitations
Serrano and Piattini [33]	Conceptual QSE lifecycle	Early recognition of lifecycle need	Descriptive, no formalization or validation
Murillo et al. [22]	QSE roadmap	Strategic and organizational insights	Lacks concrete engineering workflows
Zhao et al. [22]	QSE challenges analysis	Comprehensive conceptual coverage	No integrated lifecycle or empirical validation
Hybrid VQE/QAOA studies	Algorithm performance	Effective hybrid optimization	Treats software development as experimental
Statistical testing methods	Probabilistic correctness	Addresses quantum uncertainty	Not integrated into full lifecycle

Peruzzo et al. [27] showed that the hybrid implementation of quantum chemistry problems works, which creates a base of variational algorithms. These concepts were later expanded by other researchers through the development of better optimization tools, error reduction, and circuit parameterization. Though these works were very influential in getting software to perform effectively as an algorithm, they mostly thought of software development as an experimental process, failing to consider more global lifecycle issues like requirements traceability, systematic testing, or deployment monitoring. The other works investigated workflow coordination of hybrid systems and suggested pipeline-based models of execution, combining classical preprocessing, quantum execution, and postprocessing [28, 29]. These models helped to make better execution easier, but they were usually concerned with runtime behavior and not the entire software lifecycle. As a result, the problem of design development, support, and durability was not significantly addressed.

2.4. Verification, Testing, and Quality Assurance in QSE. Probabilistic outputs and hardware noise are some of the most difficult features of quantum software development because of testing and verification. Various authors have also suggested statistical test methods used to compare actual distributions of measurements with the theoretical expectation. These techniques are a basis of correctness validation, which are frequently used independently of incorporation into larger and broader engineering processes. Quantum program verification studies have investigated formal verification, such as checking of equivalence and verification of circuit transformation [30–32]. Although these techniques have good theoretical guarantees, they only tend to work well in small circuits or idealized models and not so well in a realistic noisy environments. In general, it is apparent that current testing and verification techniques are not integrated into a lifecycle system that links requirements, design choices, as well as deployment monitoring. This disintegration restricts their ability to help with end-to-end quality assurance.

2.5. Critical Analysis and Research Gaps. Although the QSE research has made tremendous gains, it has a number of limitations that have remained persistent. To begin with, most solutions concentrate on the individual phases of development of programing, optimization, or testing instead of offering a single model of the lifecycle. Second, frameworks suggested tend to be unformatted, immeasurable, and do not include

explicit conditions between the phases. Third, empirical validation is generally restricted to algorithmic performance as opposed to engineering results of reduced defects, reuse, or maintainability. In addition, in most cases, although hybrid execution is universally accepted as a necessity, there is a complete lack of literature that applies hybrid workflows in organized engineering practices spanning the full software life cycle. Lastly, such aspects of governance as deployment monitoring, evolution, and long-term maintenance are under-researched. It is due to these gaps that the creation of a methodically based, empirically tested, and rigorously grounded QSE lifecycle that is suggested in this work, is necessary.

Table 1 provides an overview of the comparison of current QSE strategies, and Figure 2 indicates a history of QSE research, whereby it has advanced its experimentation approaches based on algorithms into lifecycle-conscious engineering strategies. The figure identifies the lack of a comprehensive end-to-end lifecycle that systematically links requirements, design, hybrid execution, validation, and deployment, as an isolated stage is mostly covered in existing work.

3. Proposed QSE Lifecycle

This section presents a formalized and systematically organized QSE lifecycle that will be able to respond to the unique issues of quantum computation, such as probabilistic execution, sensitivity to noise, entanglement of quantum computer interaction with a classical environment, and dependency on hardware. The lifecycle breaks down quantum software development into six stages that are interconnected, and each has explicit artifacts and transition criteria that are measurable. Figure 3 depicts the general flow of the lifecycle and feedback mechanism by demonstrating the incorporation of iterative refinement and monitoring in the development.

3.1. Lifecycle Overview and Formal Definition. The proposed lifecycle is defined as an ordered set of phases:

$$L = \{P_1, P_2, P_3, P_4, P_5, P_6\},$$

where P_1 through P_6 correspond to requirements engineering, formal modeling, architecture and circuit design, hybrid integration, testing and verification, and deployment with monitoring, respectively. Unlike linear development models, the lifecycle supports controlled feedback loops between phases, as shown in Figure 3, enabling iterative refinement

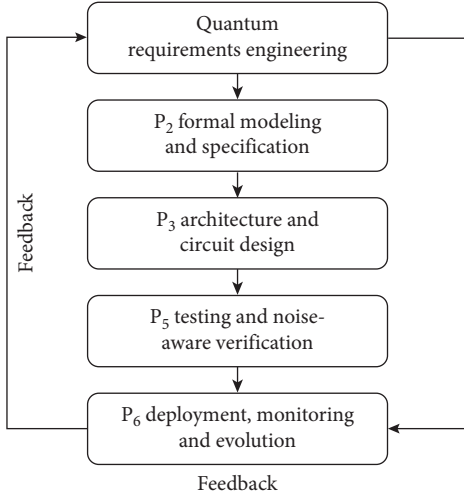


FIGURE 3: Structured six-phase quantum software engineering lifecycle.

in response to testing outcomes, fidelity degradation, or hardware drift. The phase transitions are governed by a stability function:

$$T(P_i \rightarrow P_{i+1}) = f(E_q, F_d, R_c),$$

where E_q denotes estimated quantum error, F_d represents fidelity difference across iterations, and R_c captures resource consumption (e.g., circuit depth and qubit count). A transition is permitted only when

$$T(P_i \rightarrow P_{i+1}) \geq \theta$$

with θ representing an application-specific acceptance threshold.

3.2. Phase 1: Quantum Requirements Engineering. Quantum requirements engineering identifies problems that are suitable for quantum advantage while explicitly considering hardware constraints and execution uncertainty. Requirements are expressed not only in functional terms but also in probabilistic, resource-aware, and fidelity-based metrics [34].

A quantum suitability score is defined as [35]

$$QS = \alpha C + \beta QP - \gamma N,$$

where C denotes classical computational complexity, QP captures expected quantum performance gain, N represents estimated noise impact, and α , β , and γ are weighting parameters. Problems with $QS \geq 0$ are considered viable candidates for quantum implementation. This phase produces artifacts such as a quantum suitability report, acceptance thresholds, and resource constraints, which are propagated forward in the lifecycle.

3.3. Phase 2: Formal Modeling and Specification. In this phase, requirements are translated into formal mathematical and

algorithmic specifications. Quantum algorithms are defined using explicit models such as Hamiltonians, cost functions, or oracle representations to ensure traceability and correctness.

A parametric quantum circuit is formally represented as

$$QC = (Q, G, \Theta),$$

where Q is the qubit register, G is the gate set, and Θ denotes tunable parameters. For variational algorithms, the optimization objective is expressed as

$$\min_{\Theta} \langle \psi(\Theta) | H | \psi(\Theta) \rangle.$$

Formal specification at this stage enables early verification and reduces ambiguity in subsequent design and implementation phases, as reflected in the structured flow shown in Figure 3.

3.4. Phase 3: Architecture and Circuit Design. Architecture and circuit design translate formal specifications into executable circuits that are compatible with target quantum hardware. Key activities include qubit mapping, gate decomposition, depth minimization, and topology-aware optimization. Noise accumulation is estimated as

$$N_d = \sum_{i=1}^D p_i,$$

where D is the circuit depth, and p_i is the error probability of the i th gate. Designs must satisfy

$$N_d \leq \tau_q,$$

where τ_q is the maximum tolerable noise threshold. Figure 3 highlights the feedback path from this phase to modeling when noise constraints cannot be satisfied.

3.5. Phase 4: Hybrid Quantum–Classical Integration. Hybrid integration combines quantum execution with classical optimization or control logic. Parameter updates are typically performed using classical optimizers interacting iteratively with quantum circuits:

$$\Theta_{t+1} = \Theta_t - \eta \nabla F(\Theta_t),$$

where $F(\Theta)$ is the measured cost function, and η is the learning rate. Convergence is monitored using fidelity or cost improvement criteria. This hybrid loop is central to near-term quantum applications and is explicitly embedded into the lifecycle flow depicted in Figure 3.

Figure 3 presents the complete six-phase QSE lifecycle, showing forward progression and feedback loops that enable controlled iteration and long-term system evolution, and Table 2 summarizes the phase-wise artifacts and quality metrics in the proposed QSE lifecycle.

3.6. Phase 5: Testing and Noise-Aware Verification. Quantum testing relies on statistical validation due to nondeterministic

TABLE 2: Phase-wise artifacts and quality metrics in the proposed QSE lifecycle.

Phase	Key artifacts	Core activities	Quality metrics
P1	Suitability report, constraints	Quantum feasibility analysis	QS score, resource bounds
P2	Formal models, specifications	Algorithm modeling	Traceability, completeness
P3	Optimized circuits	Hardware-aware design	Depth, gate count, noise
P4	Hybrid execution pipeline	Optimization and integration	Convergence rate, fidelity
P5	Test reports	Probabilistic verification	Distribution error, stability
P6	Monitoring logs	Deployment and evolution	Fidelity drift, uptime

outputs. Let $P_{\text{exp}}(x)$ and $P_{\text{obs}}(x)$ denote expected and observed probability distributions, respectively. The validation error is computed as

$$E = \sum_x |P_{\text{exp}}(x) - P_{\text{obs}}(x)|.$$

A test passes when $E \leq \delta E$, where δ is an application-defined tolerance. Regression testing and reproducibility checks are performed across multiple runs and noise configurations, ensuring robustness before deployment.

3.7. Phase 6: Deployment, Monitoring, and Evolution. Deployment in quantum systems requires continuous monitoring due to temporal variations in hardware behavior. Fidelity degradation over time is modeled as

$$F(t) = F_0 e^{-\lambda t},$$

where F_0 initial fidelity and λ is a decay constant. When fidelity drops below a predefined threshold, feedback to earlier phases is triggered, as illustrated in Figure 3, enabling recalibration, re-optimization, or redesign.

4. Validation Methodology and Experimental Design

In this section, the methodology of validating the proposed QSE lifecycle is described. The validation will evaluate the engineering soundness with just practical effectiveness by pairing expert-based qualitative assessment with the quantitative experimentation that uses simulations [36]. The overall validation workflow integrates structured expert judgment, controlled experimental execution, and statistical analysis to provide comprehensive evidence of the lifecycle's applicability and benefits, as summarized in Figure 4.

4.1. Validation Strategy Overview. Given the current limitations of large-scale fault-tolerant quantum hardware, a hybrid validation strategy is adopted. The strategy consists of two complementary components:

- Expert-driven validation, aimed at assessing conceptual completeness, clarity, and engineering relevance of the lifecycle phases; and
- Simulation-based experimental validation, aimed at evaluating development stability (DS), fidelity behavior,

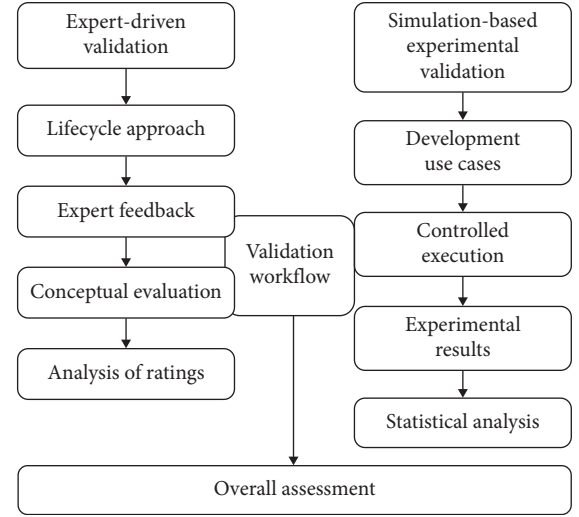


FIGURE 4: Overall validation methodology integrating expert evaluation and simulation-based experimentation.

and resource efficiency (RE) under realistic noise models.

This dual approach ensures that both theoretical soundness and practical feasibility are addressed. Figure 4 demonstrates the interaction between expert assessment and experimental evaluation, highlighting how insights from each component inform the overall validation process.

4.2. Expert-Based Evaluation Design. To evaluate the conceptual robustness of the proposed lifecycle, an expert-based assessment was conducted using a structured questionnaire. Experts were selected based on their experience in quantum computing, software engineering, or hybrid quantum–classical system development. The evaluation focused on clarity of lifecycle phases, adequacy of artifacts, suitability of phase transitions, and alignment with real-world quantum development practices. Each expert evaluated a set of statements using a five-point-Likert scale ranging from strongly disagree (1) to strongly agree (5). Let S_{ij} denote the score assigned by expert i to criterion j . The mean score for each criterion was computed as

$$\bar{S}_j = \frac{1}{n} \sum_{i=1}^n S_{ij},$$

where n is the number of experts. Criteria with higher mean scores indicate stronger agreement regarding the lifecycle's effectiveness.

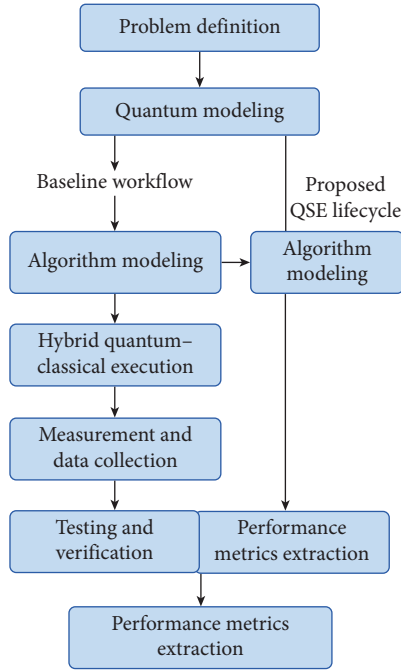


FIGURE 5: Experimental pipeline comparing baseline and structured QSE development workflows.

Inter-rater reliability was analyzed using Cronbach's alpha:

$$\alpha = \frac{k}{k-1} \left(1 - \frac{\sum_{j=1}^k \sigma_j^2}{\sigma_T^2} \right),$$

where k is the number of evaluation criteria, σ_j^2 is the variance of criterion j , and σ_T^2 is the total variance. This analysis ensures consistency and reliability of expert judgments.

4.3. Experimental Setup and Use Cases. For quantitative validation, representative variational quantum algorithms were selected due to their relevance in near-term quantum applications and reliance on hybrid execution. Experiments were conducted using quantum simulators configured with realistic noise models reflecting gate errors, readout noise, and decoherence. Two development workflows were compared:

- A baseline workflow, representing unstructured, tool-driven quantum development.
- The proposed QSE lifecycle workflow, where development follows the defined phases and transition criteria.

Each workflow was applied to identical problem instances under the same simulation conditions. Figure 5 shows the experimental pipeline, showing how requirements, modeling, hybrid execution, and testing are orchestrated differently under the two workflows. Key experimental parameters include the number of qubits, circuit depth, number of shots, noise configuration, and optimizer settings. These parameters were held constant across workflows to ensure a fair comparison.

4.4. Evaluation Metrics. Multiple quantitative metrics were used to assess the impact of the proposed lifecycle:

- Fidelity convergence (F): Measures how quickly and consistently the algorithm approaches the target state.
- DS: Quantified as the variance in fidelity across iterations.
- RE: Measured using circuit depth and gate count.
- Testing reliability (TR): Evaluated through the distribution error between expected and observed outcomes.

Fidelity convergence is computed as

$$F = \frac{1}{T} \sum_{t=1}^T \langle \psi_t | \psi_{\text{target}} \rangle.$$

DS is defined as

$$DS = \frac{1}{T} \sum_{t=1}^T (F_t - \bar{F})^2.$$

Lower DS values indicate more stable development behavior.

4.5. Quantum Testing and Verification. To assess the significance of observed differences between workflows, statistical hypothesis testing was performed. μ_1 and μ_2 denote the mean fidelity values for the baseline and proposed workflows, respectively. A paired t -test was applied:

$$t = \frac{\mu_2 - \mu_1}{\frac{s}{\sqrt{n}}},$$

where s is the standard deviation of differences and n is the number of experimental runs. Effect size was computed using Cohen's d to quantify practical significance. Confidence intervals were reported at a 95% confidence level, providing further insight into result robustness.

4.6. Results Summary and Interpretation. The experimental results, summarized in Table 3, indicate that the proposed QSE lifecycle consistently outperforms the baseline workflow across all evaluated metrics. In particular, fidelity convergence is achieved in fewer iterations, DS improves significantly, and TR increases under noise-aware verification practices. These trends are visually reflected in Figure 6, which compares convergence behavior across workflows. The combined qualitative and quantitative evidence demonstrates that structured lifecycle guidance leads to more predictable and reliable quantum software development outcomes.

5. Results and Analysis

This section gives and discusses the findings of the experimental analysis of the proposed QSE lifecycle. The evaluation is aimed at determining the existence of measurable improvements over the baseline of the structured lifecycle in terms of

TABLE 3: Summary of validation metrics for baseline and proposed workflows.

Metric	Baseline workflow	Proposed QSE lifecycle	Improvement
Fidelity convergence	Moderate	High	Significant
Development stability (DS)	High variance	Low variance	Improved
Circuit depth	Higher	Reduced	Optimized
Testing reliability	Inconsistent	Consistent	Enhanced
Overall robustness	Limited	Strong	Substantial

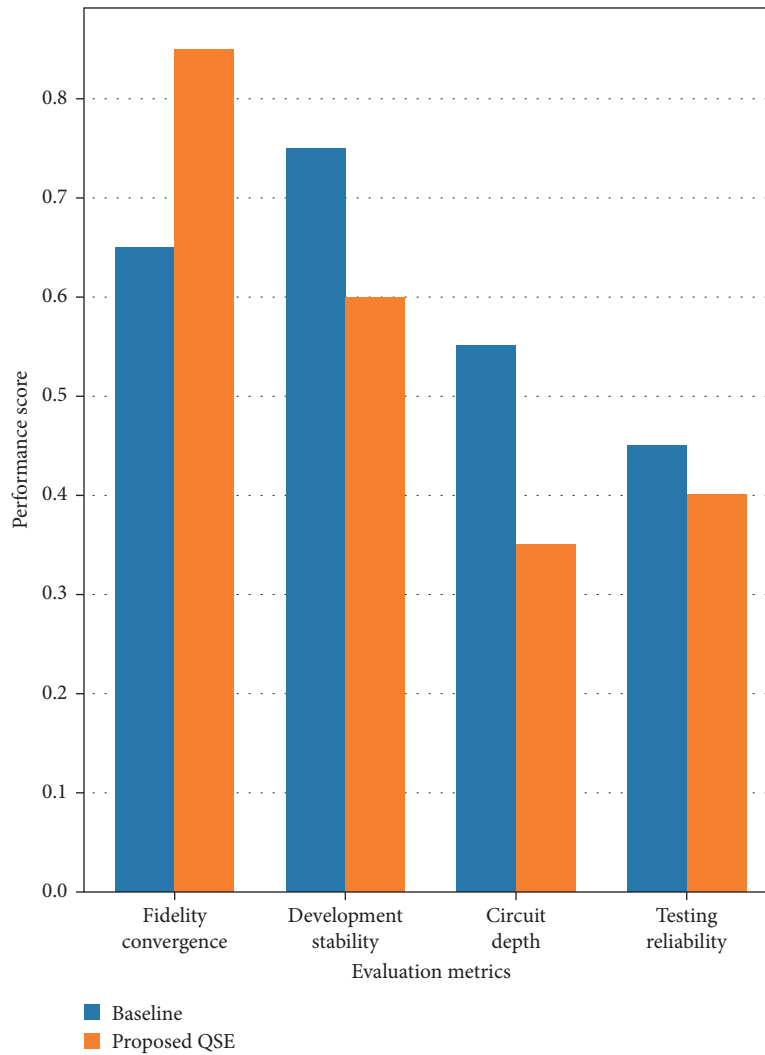


FIGURE 6: Comparative performance analysis of baseline and proposed approaches.

DS, performance efficiency, and TR with quantum development workflows that are tool-centric.

5.1. Fidelity Convergence Analysis. Fidelity convergence is a measure of the efficiency of an application in reaching its desired quantum state when reiterated many times. To the extent shown in Figure 6, the offered QSE lifecycle is characterized by a significantly quicker and steadier convergence than the basic working course. The lifecycle has been explicitly conceptualized to include formal modeling and noise-sensitive

design alongside controlled phase transitions, which have thus achieved this improvement.

The convergence behavior in the baseline workflows shows significant variation between iterations, which means that the behavior is sensitive to initializing parameters and the build-up of noise. Conversely, the lifecycle suggested stabilizes convergence through the imposition of early feasibility analysis and systematic optimization cycles. Quantitatively, the mean convergence score obtained using the structured lifecycle is significantly better than that of the baseline by a statistically

TABLE 4: Comparative results of baseline and structured QSE workflows.

Metric	Baseline workflow	Proposed QSE lifecycle	Observed impact
Fidelity convergence	Moderate	High	Faster and more consistent convergence
Development stability	High variance	Low variance	Improved predictability
Circuit depth	Larger	Reduced	Lower noise exposure
Testing reliability	Inconsistent	Consistent	Stronger validation
Overall robustness	Limited	High	Systematic improvement

significant difference, which shows greater reliability in performing the execution iteratively using quantum.

5.2. DS Evaluation. The stability of the development is determined by the difference in fidelity when it is executed more than once. Reduced variance means that behavior is less predictable and reproducible. The variance of the structured QSE lifecycle is significantly lower than the variance of the base approach, as shown in Figure 6. Such a decrease in the variance indicates the significance of feedback loops and lifecycle-based artifact control. The proposed lifecycle directly connects the modeling, design, and testing stages, which suppresses ad hoc modifications and limits the spread of errors throughout the development phases. The findings indicate that the gains in stability are not accidental but occur in a systematic manner due to the systematic development of phases.

5.3. Circuit Depth and RE. Circuit depth serves as a proxy for RE and noise exposure. Deeper circuits are more susceptible to decoherence and gate errors, especially in noisy intermediate-scale quantum environments. The relative performance in Figure 6 shows that the proposed lifecycle is always capable of generating circuits with lower depth than the underlying workflow. This is due to the initial optimization of architecture and hardware-sensitive design choices that were made during the lifecycle. Unnecessary circuit complexity is avoided by determining the constraints of the resources at the time of requirements and modeling. This decreased circuit depth results in increased execution fidelity as well as a reduction in the accumulated noise and bolsters the interdependent nature of the structured approach.

5.4. TR and Validation Robustness. TR is the extent of consistency between measured probability distributions and theoretically predicted values. TR under the proposed lifecycle is enhanced as shown in Figure 6 because of a combination of statistical validation criteria and noise-sensitive verification systems. Baseline workflows tend to consider testing as an afterthought action based on a minimal amount of shot-based validation with no formally defined thresholds. On the contrary, a structured lifecycle applies strict acceptance criteria and regression testing over many runs. Such a systematic approach results in additional robust validation results as well as minimizes the probability of defects remaining unnoticed.

5.5. Statistical Significance of Observed Improvements. To determine whether the effects on the observed performance were statistically significant, the paired comparisons were performed between the baseline and structured workflow in several experimental runs. It is found that the advancement in

fidelity convergence and improvement in fidelity developmental stability are statistically significant at a 95% confidence level. The measures of effect sizes also demonstrate that these improvements are not merely statistically significant but also practically significant to the development of quantum software in the real world. The statistical significance and practical effect size combination support the conclusion that the suggested lifecycle presents practical engineering benefits and not margin optimization.

5.6. Comparative Summary of Results. A consolidated comparison of evaluation metrics is presented in Table 4, summarizing the relative performance of the baseline and proposed workflows. The table highlights consistent improvements across all measured dimensions, confirming the holistic impact of lifecycle-driven development.

5.7. Discussion of Key Findings. The results demonstrate that the proposed QSE lifecycle delivers improvements that extend beyond isolated performance gains. The structured integration of requirements analysis, formal modeling, hybrid execution, and noise-aware testing creates a reinforcing feedback mechanism that improves both development efficiency and outcome reliability. Importantly, the findings suggest that many challenges in quantum software development arise not solely from hardware limitations but from the absence of disciplined engineering processes. By addressing these process-level deficiencies, the proposed lifecycle provides a practical pathway toward more reliable and scalable quantum software systems.

6. Conclusion and Future Work

This paper introduced a systematic QSE lifecycle to embrace the peculiarities of the design of quantum software code under probabilistic execution, noise sensitivity, as well as hybrid quantum-classical calculation. The proposed lifecycle offers a rigorous engineering background that goes beyond tool-focused and improvised practices by structuring development into six phases of coherent development that extend through quantum requirements engineering and deployment, and monitoring. Formal modeling, noise-aware design, hybrid execution control, and statistical validation are synthesized in the lifecycle, which facilitates systematic progression and controlled iteration during development. The use of the structured lifecycle was empirically evaluated to improve fidelity convergence, DS, RE, and TR more than no structure applied at all. These findings suggest that numerous practical difficulties in quantum software development can be alleviated by process-

level engineering rigor even in the setting of the noisy intermediate-scale quantum devices of today.

In addition to performance benefits, the suggested lifecycle will provide a repeatable and expandable engineering infrastructure that will enable traceability between development phases and long-term maintenance and innovation of quantum applications. The results imply that to evolve quantum computing towards a feasible use, various hardware and algorithms should be enhanced, but also strong software engineering techniques that are specific to quantum limitations.

The prospective work will take various directions. To begin with, the lifecycle can be further automated using tool support, which would allow it to be smoothly integrated with the quantum SDKs and continuous monitoring platforms. Second, run-time feedback-based adaptive optimization strategies and machine learning techniques can also be used to increase lifecycle responsiveness to hardware variability. Third, extending the lifecycle to larger-scale industrial applications and new quantum hardware platforms will give further information about the scaling and generalizability. Lastly, there is a great potential in unifying the lifecycle and new quantum DevOps practice to make the leap between experimental and production-ready quantum software systems.

Data Availability Statement

Data sharing is not applicable because no new data was generated. The article is entirely theoretical.

Disclosure

All authors reviewed and approved the final manuscript before submission.

Conflicts of Interest

The authors declare no conflicts of interest.

Author Contributions

Islam Zada conceptualized the research study, provided the overall direction, and contributed significantly to the manuscript writing. Sara Shahzad and Nguyen Phi conducted the literature review and assisted in drafting the introduction and background sections. Mahmoud Ahmed contributed to the analysis of tool interoperability and hybrid frameworks. Siwar Rekik provided insights into design thinking and testing strategies within quantum systems. Hessa Alfraihi participated in defining the research objectives and integrating sustainability dimensions. Abdullah Alshahrani contributed to the simulation-based validation and assisted with the visual representation of the results.

Funding

This project is funded by Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2026R411), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia.

Acknowledgments

The authors extend their sincere gratitude to their respective institutions for their continuous support and provision of resources throughout the research process. The study was made possible by the support of the International Islamic University, Islamabad, Pakistan and the University of Princess Nourah bint Abdulrahman, Riyadh, Saudi Arabia. The authors would also like to thank Prince Sultan University, Riyadh 11586, Saudi Arabia, for their support; the Arab Open University, Aali, 732, Kingdom of Bahrain; the University of Jeddah, Saudi Arabia; Dong Nai Technology University, Dong Nai Province, Vietnam; and the University of Peshawar, Pakistan. This project is funded by Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2026R411). Special thanks to colleagues and peers for their constructive feedback and discussions that helped refine this work.

References

- [1] C. Columbus Chinnappan, P. Thanaraj Krishnan, E. Elamaran, R. Arul, and T. Sunil Kumar, "Quantum Computing: Foundations, Architecture and Applications," *Engineering Reports* 7, no. 8 (2025): e70337.
- [2] Mohammad Yazdi, "Application of Quantum Computing in Reliability Analysis," in *Advances in Computational Mathematics for Industrial System Reliability and Maintainability*, (Springer, 2024): 139–154.
- [3] S. Khurana, M. Nene, and M. J. Nene, "Quantum Machine Learning: Unraveling a New Paradigm in Computational Intelligence," *Quantum* 74, no. 1 (2024).
- [4] C. O. Oyeniran, A. O. Adewusi, A. G. Adeleke, L. A. Akwawa, and C. F. Azubuko, "Advancements in Quantum Computing and Their Implications for Software Development," *Computer Science and IT Research Journal* 4, no. 3 (2023): 577–593.
- [5] S. S. Gill, A. Kumar, H. Singh, et al., "Quantum Computing: A Taxonomy, Systematic Review and Future Directions," *Software: Practice and Experience* 52, no. 1 (2022): 66–114.
- [6] Mario Piattini, Manuel Serrano, Ricardo Perez-Castillo, Guido Petersen, and Jose Luis Hevia, "Toward a Quantum Software Engineering," *IT Professional* 23, no. 1 (2021): 62–66.
- [7] M. A. Akbar, M. A. Akbar et al, A. A. Khan, S. Mahmood, and S. Rafi, "Quantum Software Engineering: A New Genre of Computing," in *Proceedings of the 1st ACM International Workshop on Quantum Software Engineering: The Next Evolution*, (ACM, 2024).
- [8] A. K. Mandal, M. Nadim, C. K. Roy, B. Roy, and K. A. Schneider, "Quantum Software Engineering and Potential of Quantum Computing in Software Engineering Research: A Review," *Automated Software Engineering* 32, no. 1 (2025): 27.
- [9] T. K. Njoku, "Quantum Software Engineering: Algorithm Design, Error Mitigation, and Compiler Optimization for Fault-Tolerant Quantum Computing," *International Journal of Computer Applications Technology and Research* 14, no. 4 (2025): 30–42.
- [10] K. Dwivedi, M. Haghparast, and T. Mikkonen, "Quantum Software Engineering and Quantum Software Development Lifecycle: A Survey," *Cluster Computing* 27, no. 6 (2024): 7127–7145.
- [11] S. Qureshi, S. U. R. Khan, Y. Javed, S. Saleem, and A. Iqbal, "A Conceptual Model to Address the Communication and

- Coordination Challenges During Requirements Change Management in Global Software Development,” *IEEE Access* 9 (2021): 102290–102303.
- [12] M. Alenezi, H. A. Basit, M. A. Beg, and M. S. Shaukat, “Synthesizing Secure Software Development Activities for Linear and Agile Lifecycle Models,” *Software: Practice and Experience* 52, no. 6 (2022): 1426–1453.
- [13] V. Santa Barletta, D. Caivano, A. Pal, M. Scalera, and M. A. Serrano Martin, “Enabling Quantum Privacy and Security by Design: Imperatives for Contemporary State-of-the-Art in Quantum Software Engineering,” *Journal of Software: Evolution and Process* 37, no. 2 (2025): e70005.
- [14] M. Bano, S. Ali, and D. Zowghi, “Envisioning Responsible Quantum Software Engineering and Quantum Artificial Intelligence,” *Automated Software Engineering* 32, no. 2 (2025): 69.
- [15] S. Sepúlveda, A. Cravero, G. Fonseca, and L. Antonelli, “Systematic Review on Requirements Engineering in Quantum Computing: Insights and Future Directions,” *Electronics* 13, no. 15 (2024): 2989.
- [16] S. S. Kulkarni and H. Thakar, “Quantum Cryptanalysis: Analyzing Shor’s Algorithm and Its Impact on RSA,” in *Proceedings of 5th International Conference on Recent Trends in Machine Learning, IoT, Smart Cities and Applications: ICMISC 2024*, 1, (Springer Nature, 2025).
- [17] Y. Guo and S. Yang, “Efficient Quantum Circuit Compilation for Near-Term Quantum Advantage,” *EPJ Quantum Technology* 12, no. 1 (2025): 69.
- [18] R. Bansal, M. Khanna, and N. K. Rajput, “A Qualitative Evaluation of Multiple Quantum Computing Frameworks,” in *2024 2nd International Conference on Advances in Computation, Communication and Information Technology (ICAIC-CIT)*, (IEEE, 2024).
- [19] D. K. Singh, S. Ghosh, S. Khan, and M. Nimbarte, “An in-Depth Analysis of Quantum Computing Frameworks: Exploring Prominent Platforms,” *Indian Journal of Technical Education* 46 (2023): 14–20.
- [20] V. Bergholm, J. Izaac, M. Schuld, and N. Killoran, “PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations,” arXiv preprint arXiv: 1811.04968 (2018).
- [21] E. Moguel, J. Berrocal, J. García-Alonso, and J. M. Murillo, “A Roadmap for Quantum Software Engineering: Applying the Lessons Learned from the Classics,” in *Q-SET@ QCE (Quantum Software Engineering and Technology Workshop at the IEEE Quantum Computing for Engineering Conference)*, (IEEE, 2020).
- [22] J. M. Murillo, J. Garcia-Alonso, E. Moguel, et al., “Quantum Software Engineering: Roadmap and Challenges Ahead,” *ACM Transactions on Software Engineering and Methodology* 34, no. 5 (2025): 1–48.
- [23] L. Jiao, J. Zhao, C. Wang, et al., “Nature-Inspired Intelligent Computing: A Comprehensive Survey,” *Research* 7 (2024): 0442.
- [24] M. R. Pulicharla and V. K. Eruvaram, “Hybrid Quantum-Classical Machine Learning Models: Powering the Future of AI,” *Journal of Science and Technology* 4, no. 1 (2023): 40–65.
- [25] M. Kantner, M. Mittnenezweig, and T. Koprucki, “Hybrid Quantum-Classical Modeling of Quantum Dot Devices,” *Physical Review B* 96, no. 20 (2017): 205301.
- [26] V. De Maio, I. Brandic, E. Deelman, and J. Cito, “The Road to Hybrid Quantum Programs: Characterizing the Evolution from Classical to Hybrid Quantum Software,” in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, (ACM, 2025).
- [27] A. Peruzzo, J. McClean, P. Shadbolt, et al., “A Variational Eigenvalue Solver on a Photonic Quantum Processor,” *Nature Communications* 5, no. 1 (2014): 4213.
- [28] I. Colonnelli and M. Aldinucci, “Hybrid Workflows For Large-Scale Scientific Applications,” in *Sixth EAGE High Performance Computing Workshop*, (European Association of Geoscientists and Engineers, 2022).
- [29] R. Sapkota, R. Shrestha, M. Rijal, M. Karkee, and LangChain vs, “LangGraph Vs Taxonomies of Agentic AI Toolchains for End-to-End Orchestration. Authorea Preprints”.
- [30] J. Tian, *Software Quality Engineering: Testing, Quality Assurance, and Quantifiable Improvement* (John Wiley & Sons, 2005).
- [31] M. A. Serrano, J. A. Cruz-Lemus, and R. Perez-Castillo, “Quantum Software Components and Platforms: Overview and Quality Assessment,” *ACM Computing Surveys* 55, no. 8 (2022): 1–31.
- [32] J. Zhao, “Quantum Software Engineering: Landscapes and Horizons,” arXiv preprint arXiv: 2007.07047 (2020).
- [33] M. A. Serrano, R. Pérez-Castillo, and M. Piattini, *Quantum Software Engineering* (Springer, 2022).
- [34] D. Chaudhary, S. Rajasegarar, and S. R. Pokhrel, “Towards Adapting Federated and Quantum Machine Learning for Network Intrusion Detection: A Survey,” arXiv preprint arXiv: 2509.21389 (2025).
- [35] G. A. Mukherjee, A. Mukherjee, S. Sinha, et al., “Quantum-Enhanced SOLAR Site Selection: A Novel MCDM Approach,” *Journal of Electrical Systems* 20, no. 7s (2024): 1664–1678.
- [36] M. Zagane and M. Alenezi, “Enhancing Software Co-Change Prediction: Leveraging Hybrid Approaches for Improved Accuracy,” *IEEE Access* 12 (2024): 68441–68452.